All-Weather Image Restoration

How We Plan to Improve DTMIR-Pro (and Why)

Pustak Pathak

Advisor: Dr. Snehasis Mukherjee

Project Update

What This Update Covers

1. A plain-language **flowchart**: how the ideas from our reading list combine into one improved pipeline
2. Where each piece comes from, what it replaces, and why it's an improvement
3. What we've started building in code
4. A quick honest check: are these actually problems worth solving?

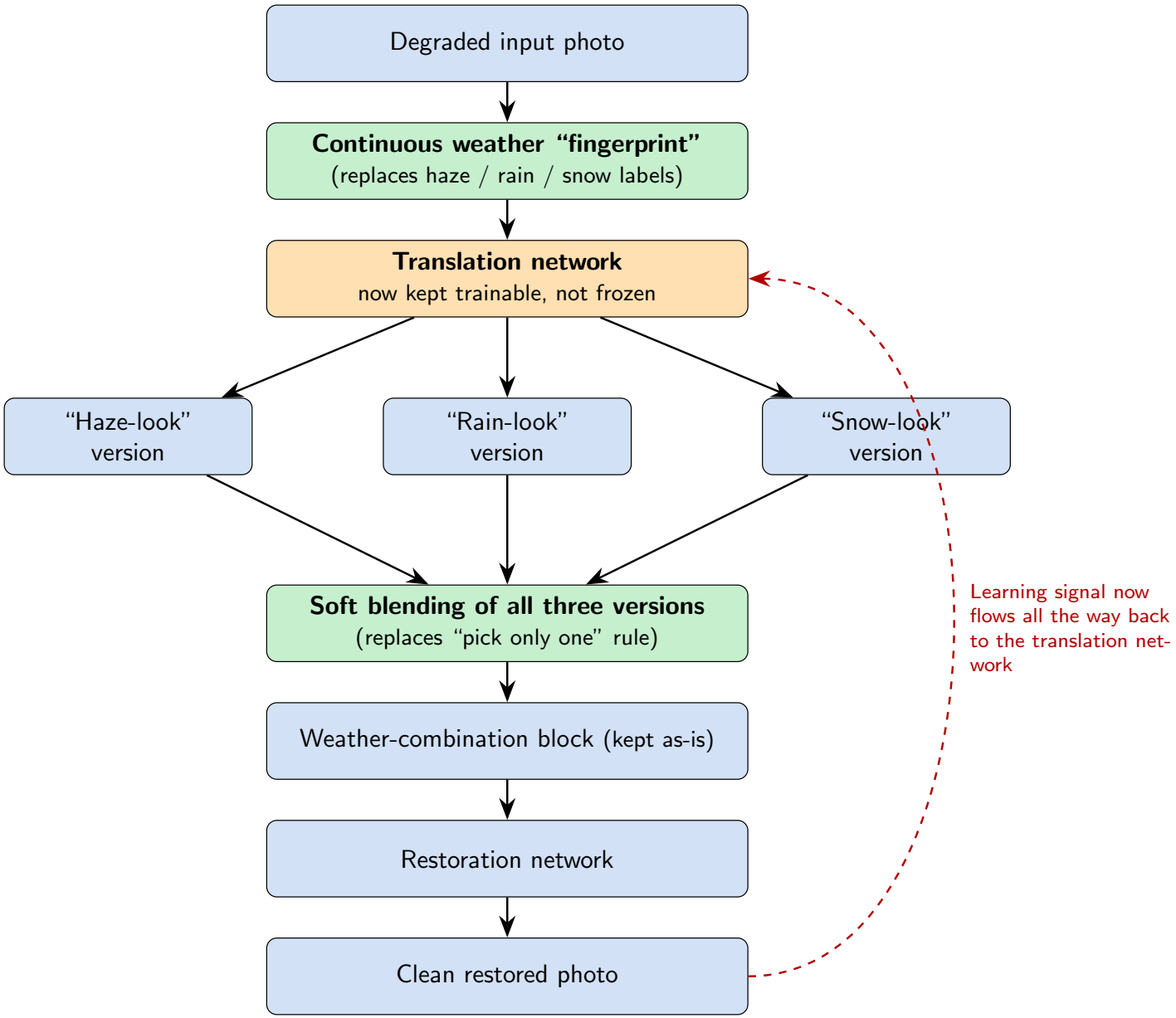
Quick Recap: What's Wrong With DTMIR-Pro Today

DTMIR-Pro (our baseline) works in two steps: it first **translates** a degraded image into a haze-look, a rain-look, and a snow-look version, then **restores** the clean image using whichever of those three looks the closest to a known weather type.

Three weak points we identified:

1. The translation step is trained once and then **frozen** — it never learns from how well the final restoration actually turns out.
2. It can only describe an image as **100% haze, 100% rain, or 100% snow** — real photos are often a mix of these.
3. It **throws away two out of three** translated versions and keeps only one, based on a rigid similarity rule.

The Big Picture: Our Improved Pipeline



How to Read the Flowchart

Unchanged	Modified	Brand new
Parts of DTMIR-Pro we are keeping exactly as they are — they already work well.	An existing DTMIR-Pro part that we're changing how it behaves (not replacing it entirely).	A component that didn't exist in DTMIR-Pro at all — newly added, inspired by another paper.

Dashed red arrow: information that now flows *backward* during training (a "feedback loop") — this did not exist in the original pipeline.

Change 1 — A "Fingerprint" Instead of a Label

What it replaces: DTMIR-Pro tags every image with a rigid label: haze, rain, *or* snow. Nothing in between.

What we're doing instead: Give every image a flexible, learned description of its weather condition — more like a fingerprint than a checkbox. This lets the network say "this photo is somewhat rainy and somewhat hazy" instead of forcing a single choice.

Where the idea comes from: The BDG paper (ICLR'26) uses a similar idea — instead of a fixed label for the type of damage in an image, it builds a rich, learned description of the damage and feeds that into its model. We're borrowing that same philosophy for weather type.

Why it matters: Real bad-weather photos are almost never purely one thing — a rainy evening often also looks hazy. A single label can't capture that; a fingerprint can.

Change 2 — Letting the Translator Learn From the Outcome

What it replaces: In DTMIR-Pro, the part of the network that creates the haze/rain/snow versions of the image is trained *once*, then locked. The later restoration step can never tell it "you got that a bit wrong, adjust."

What we're doing instead: Keep that part unlocked during training, so the quality of the final restored photo can send feedback backward and improve how the translations are made in the first place.

Where the idea comes from: This is our own proposal, but it's the same principle that shows up across all the papers we read — every one of them either avoids freezing a component or actively re-uses feedback signals (e.g. EquS combining information from two passes instead of trusting one frozen pass blindly).

Why it matters: A part of the system that never learns from its own mistakes is wasted potential — this closes that loop.

Change 3 — Blending Instead of Picking One

What it replaces: DTMIR-Pro looks at the three translated versions (haze/rain/snow look-alikes), measures which one looks least like the original photo, and keeps *only that one*. The other two are thrown away completely.

What we're doing instead: Keep all three, but assign each one a learned "how useful is this" weight, and blend them together instead of picking a winner. This is a small, ordinary neural-network layer that can be trained end-to-end, unlike the old rigid rule.

Where the idea comes from: MIRAGE (ICLR'26) shows that combining information from multiple feature streams with a learned similarity/gating step works better than keeping only one and discarding the rest.

Why it matters: Throwing away two-thirds of your information on a rigid rule is wasteful, and the rule itself has no way to improve during training.

What We Looked At but Are *Not* Using Right Now

Restora-Flow and **EquS / EquS$^+$** both improve the *generative sampling process* of diffusion- or flow-matching-based restoration models.

DTMIR-Pro's restoration step is a direct, single-pass network — it isn't a diffusion or flow model, so these two papers' core tricks don't plug in without a much larger redesign.

We're parking them as a possible *future* direction, not using them in this update.

This is intentional: it keeps the current update focused and realistic instead of combining every paper into one giant redesign.

Summary Table

Change	**Replaces**	**Inspired by**	**Benefit**
Continuous weather fingerprint	Fixed haze/rain/snow label	BDG (ICLR'26)	Handles mixed real-world weather
Unfrozen translator	Frozen, one-time-trained translator	Our own proposal	Translator improves alongside restoration
Soft blending of streams	Hard "pick one, discard two" rule	MIRAGE (ICLR'26)	No information thrown away; trainable

Considered but deferred: Restora-Flow, EquS/EquS^{+} (both assume a diffusion/flow-based restorer, which DTMIR-Pro is not).

Code: Where We Stand

- DTMIR-Pro's own GitHub repository has **no released code** — so we are building everything from scratch, using the paper as a blueprint.
- We've started laying out the project as separate, swappable pieces (image translator, weather fingerprint, blending step, weather-combination block, restorer) so each of our three changes can be built and tested on its own before wiring them together.
- Next steps: get a basic version of the original DTMIR-Pro architecture running end-to-end first, *then* swap in our three changes one at a time so we can measure each one's effect separately.

Thank you — Questions / Discussion